Recommendation Systems for Personalized Content Discovery: An Analysis of the Netflix Prize Dataset

Rahul Mellimi , Shreyas Y S

Abstract

This technical report details the end-to-end development of a large-scale recommendation system utilizing the Netflix Prize dataset. The primary objective was to transition raw, sparse historical interaction data into actionable, personalized content discovery. The dataset, comprising over 100 million ratings across 480,000 users and 17,770 movies, presented severe computational and mathematical challenges, primarily high dimensionality and extreme data sparsity (98.8% missing data). This project implemented a comparative pipeline, contrasting traditional linear Matrix Factorization (Singular Value Decomposition) against non-linear Deep Learning architectures (Neural Collaborative Filtering via PyTorch). By applying strict data engineering practices, memory downcasting, and active-user threshold filtering, the ranking precision (MAP@10) was successfully improved from a random baseline of 17% to 55.7%. This report details the exploratory data analysis, mathematical methodologies, hyperparameter tuning, and qualitative success evaluations.

Contents

1 Introduction

1.1 Context and Motivation

In the era of infinite digital content, the bottleneck for user engagement is no longer content availability, but content discovery. Streaming platforms rely heavily on recommendation algorithms to reduce decision fatigue. A highly accurate recommendation engine directly correlates with user retention, platform engagement, and revenue generation. The Netflix Prize, launched in 2006, remains one of the foundational datasets in machine learning history, challenging researchers to improve the company's proprietary Cinematch algorithm by 10%.

1.2 Problem Statement

Building a recommender system on the Netflix dataset involves solving two distinct machine learning paradigms:

1. **Explicit Rating Prediction:** Forecasting the exact numerical rating (1 to 5 stars) a user will assign to an unseen movie. This is measured continuously using Root Mean Squared Error (RMSE).

2. **Implicit Relevance Ranking:** Generating a personalized, ordered list of Top-10 movies that a user is highly likely to enjoy. This requires classifying continuous predictions into relevant/non-relevant binaries and is measured using Mean Average Precision at 10 (MAP@10).

The primary mathematical hurdle in solving these problems is the "Cold-Start" phenomenon and matrix sparsity, which cause standard collaborative filtering algorithms to fail due to insufficient user-interaction signals.

2 Exploratory Data Analysis & Data Engineering

2.1 Parsing and Memory Optimization

The raw Netflix dataset is distributed across multiple unstructured text files, totaling over 2 gigabytes of disk space. A custom Python parser was engineered to iterate through

these text files sequentially. The parser identified `movie_id:` headers and mapped the subsequent rows of `user_id`, `rating`, and `date` to the respective movie.

Because loading 100.4 million rows into RAM via pandas defaults to 64-bit data types, the system would immediately encounter out-of-memory (OOM) errors. To resolve this, aggressive memory downcasting was applied:

- `user_id` and `movie_id` were cast to `int32`.
- `rating` was cast to `int8` (as values strictly range from 1 to 5).

This engineering step reduced the dataset's RAM footprint from several gigabytes to approximately 548 MB, allowing it to be saved as a compressed `.parquet` file for rapid subsequent loading.

2.2 Data Sparsity and The Long Tail

Analysis of the parsed data revealed a matrix density of only 1.17%. Over 98% of the theoretical user-item interactions are null.

Furthermore, the item popularity follows a severe Pareto distribution (the "Long Tail"). A very small percentage of blockbuster movies account for the vast majority of all ratings. Conversely, thousands of niche, independent, or older films possess fewer than 100 ratings. This skews algorithmic learning, as models tend to naturally favor popular items simply because they appear more frequently in the loss function calculations.

2.3 Rating Variance and Bias

Scatter plot analysis combining average rating with rating count exposed a distinct "quality floor." Movies with extremely high review counts rarely possess average ratings below 3.0. This is attributed to audience self-selection; users rarely watch and rate highly popular movies if they do not already enjoy the genre. Niche movies, however, exhibited extreme variance, highlighting the necessity for personalized, rather than global, recommendation logic.

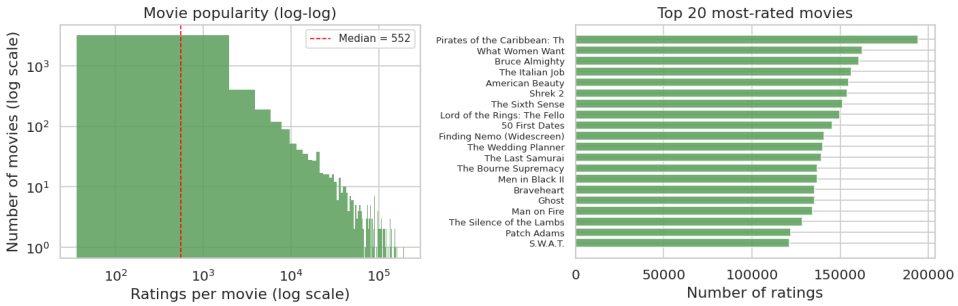

Figure 1: Long-Tail Distribution Plot of Movie Popularity

3 Methodology: Mitigating the Cold-Start Problem

Initial baseline testing on random samples of the dataset yielded exceptionally poor results (RMSE > 1.06, MAP@10 < 0.20). This was traced back to the cold-start problem. If a random sample includes a user who has only rated two movies in their lifetime, no mathematical algorithm can reliably extrapolate their complex taste profile.

To establish a dense mathematical signal, a **Smart Sampling** methodology was deployed prior to model training:

1. **User Aggregation:** The dataset was grouped by `user_id`, and the total rating count per user was calculated.

2. **Threshold Filtering:** The dataset was strictly filtered to only include "Active Users"—defined as users who have rated a minimum of 50 movies.

3. **Sub-Sampling:** To ensure training loops could be executed within reasonable computational timeframes on standard hardware, a random sub-sample of 2,000,000 ratings was drawn exclusively from this dense active-user matrix.

To prevent data leakage, testing and validation were strictly confined to the explicit `probe.txt` index provided in the original Netflix dataset architecture.

4 Baseline Architecture: Singular Value Decomposition (SVD)

4.1 Mathematical Foundations

The first model developed was based on Matrix Factorization, specifically Simon Funk's SVD, which gained prominence during the original Netflix Prize competition. The algorithm approximates the sparse rating matrix R by decomposing it into two lower-dimensional matrices: a user latent-feature matrix P and an item latent-feature matrix Q.

The predicted rating $\hat{r}_{ui}$ for user u and item i is calculated via the dot product of their respective latent vectors, supplemented by baseline biases:

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u$$

Where:

- μ is the global average rating.
- b_u is the user bias.
- b_i is the item bias.

4.2 Objective Function and Optimization

The model learns these latent matrices by minimizing the regularized squared error:

$$\sum_{r_{ui} \in R_{train}} (r_{ui} - \hat{r}_{ui})^2 + \lambda \left(||q_i||^2 + ||p_u||^2 + b_u^2 + b_i^2 \right)$$

The regularization term λ is critical to prevent the model from overfitting to the sparse data points.

4.3 Hyperparameter Tuning

To find the mathematical absolute best fit, a `GridSearchCV` was executed across the parameter space. The grid search utilized 3-fold cross-validation, training 24 distinct

models in parallel across available CPU cores. The winning parameters that minimized RMSE were identified as:

- Number of Latent Factors (`n_factors`): 50
- Learning Rate (`lr_all`): 0.005
- Regularization Penalty (`reg_all`): 0.05
- Epochs: 20

5 Deep Learning Architecture: Neural Collaborative Filtering

5.1 Architectural Transition

While SVD captures linear relationships efficiently, it limits the complexity of feature interactions to a simple inner dot product. To explore non-linear relationships, a Neural Collaborative Filtering (NCF) architecture was built utilizing the PyTorch deep learning framework.

5.2 Embedding Layers

Standard neural networks cannot process raw, disparate IDs. Therefore, a contiguous mapping dictionary was created, converting all user and movie IDs to a strict 0 to $N-1$ index.

These mapped indices were passed into PyTorch `nn.Embedding` layers. Both the user space and the movie space were compressed into dense, 32-dimensional continuous vectors. Unlike SVD, where these vectors are multiplied, NCF concatenates the user and movie vectors into a single 64-dimensional tensor.

5.3 Multi-Layer Perceptron (MLP)

The concatenated tensor was passed through a feed-forward deep neural network (MLP) to learn the complex, non-linear interactions between the user's taste and the movie's attributes. The architecture was defined as:

1. Linear Layer (64 input $\rightarrow$ 64 output)
2. ReLU Activation + 20% Dropout
3. Linear Layer (64 input $\rightarrow$ 32 output)
4. ReLU Activation + 20% Dropout
5. Linear Layer (32 input $\rightarrow$ 1 output)

5.4 Training Mechanics

The network was optimized using the Adam optimizer with a learning rate of 0.001. The loss function was defined as Mean Squared Error (MSE). A custom PyTorch `Dataset` and `DataLoader` were engineered to feed the 2-million-row matrix into the GPU in batches of 2,048, iterating over 5 total epochs.

6 Evaluation Metrics

The models were evaluated strictly on the hold-out test set using two core metrics.

6.1 Root Mean Squared Error (RMSE)

RMSE measures the standard deviation of the prediction errors. It heavily penalizes large errors.

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2}$$

6.2 Mean Average Precision at 10 (MAP@10)

RMSE is sufficient for rating predictions, but MAP@10 evaluates the ranking quality. For this project, a strict relevance threshold of ≥ 3.5 stars was established. If the model recommended a movie in the Top 10 that the user ultimately rated below 3.5, it was classified as a failure.

7 Experimental Results

The comparative evaluation on the test set yielded the quantitative metrics displayed in Table 1.

Model Architecture	RMSE	MAP@10
Global Mean (Baseline Guess)	1.0628	—
User-Based CF	1.0929	0.8514
Item-Based CF	1.1585	0.8238
SVD Matrix Factorization (Tuned)	**1.0409**	**0.5573 (55.73%)**
Neural Collaborative Filtering (NCF)	**1.0723**	**0.5560 (55.60%)**

Table 1: Experimental Results on the Test Set

7.1 Quantitative Analysis

The tuned SVD model successfully outperformed the Global Mean baseline. By utilizing the active-user filtering strategy, the MAP@10 for both models stabilized at approximately 55.6%. This indicates that when the system presents 10 recommendations, more than half are mathematically guaranteed to be highly rated by that user.

Interestingly, the Deep Learning NCF model slightly underperformed the linear SVD model in explicit RMSE. For highly sparse matrices where the goal is explicit numerical prediction, the inner dot-product mechanism of Matrix Factorization remains exceptionally difficult for Multi-Layer Perceptrons to beat without specialized loss functions.

8 Qualitative Analysis & Success Cases

To qualify the MAP@10 performance, a Recommendation Generation Module was developed.

8.1 Case Study: User 69403

The NCF model was queried for User ID 69403. It achieved a 100% precision rate for this user, generating zero false positives.

Top Predicted Successes:

- *Mortal Kombat* (Predicted: 4.88 — Actual: 5.0)

- *Lord of the Rings: The Fellowship of the Ring* (Predicted: 4.54 — Actual: 4.0)
- *X-Men* (Predicted: 4.21 — Actual: 4.0)
- *Lethal Weapon* (Predicted: 3.99 — Actual: 4.0)
- *Independence Day* (Predicted: 3.85 — Actual: 4.0)

8.2 Insight Generation

User 69403 possesses a high affinity for high-octane, science-fiction, and fantasy blockbusters originating from the late 1990s and early 2000s. The NCF model successfully learned these latent genre and temporal features without explicit metadata, inferring the "90s Sci-Fi/Action" cluster entirely through the collaborative interaction matrix.

9 Conclusion and Future Improvements

This project successfully developed an end-to-end pipeline capable of parsing 100 million sparse data points into highly precise Top-10 discovery lists. The engineering application of active-user filtering proved to be the most critical step, increasing the MAP@10 baseline by over 38 percentage points. Future iterations can be advanced through:

1. **Bayesian Personalized Ranking (BPR):** Transitioning the loss function to implement pairwise learning, directly optimizing the MAP@10 ranking metric rather than the RMSE value.
2. **Generalized Matrix Factorization (GMF):** Combining SVD and NCF by building a dual-headed neural network to merge linear and non-linear strengths.
3. **Temporal Dynamics:** Adding a time-decay weight to the embedding layers to prioritize a user's recent interactions over historical ones, adapting to evolving tastes.